API Help Book

Ride-Sharing Backend

Complete Guide: Every API, Every Flow, From Rider Registration to Trip Completion

Service	Port	Purpose
API Gateway	8080	Single entry point — JWT auth + Redis rate limiting
User Service	8081	Rider registration, login, profile
Driver Service	8082	Driver register, login, location, availability
Ride Service	8083	Ride lifecycle: request → accept → start → complete
Payment Service	8084	Stripe payment, circuit breaker, idempotency
Pricing Service	8085	Fare estimate, surge multiplier per zone
Notification Svc	8086	Kafka listener — sends push/SMS/email notifications
Config Server	8888	Centralised config for all services
Eureka Server	8761	Service discovery + load balancing

All requests go through the API Gateway on port 8080. The Gateway validates your JWT token and forwards X-User-Id and X-User-Role headers to downstream services. Register/login endpoints are public — all others require Authorization: Bearer <token>.

Section 1 — Complete Execution Flow (Rider to Trip Completed)

This is the full journey — every step in order from a rider opening the app to a trip completing and payment being processed.

1 **Rider Registers**

POST /api/users/register → user-service saves to PostgreSQL, returns JWT token

2 **Rider Logs In (subsequent sessions)**

POST /api/users/login → validates BCrypt password, returns fresh JWT (24h expiry)

3 **Driver Registers**

POST /api/drivers/register → driver-service saves to PostgreSQL, returns JWT token immediately

4 **Driver Logs In (subsequent sessions)**

POST /api/drivers/login → validates password, returns fresh JWT token

5 **Driver Goes Online**

POST /api/drivers/online → sets DriverStatus=ONLINE, registers GPS in Redis GeoSet via GEOADD

6 **Driver Updates Location (every 5s)**

POST /api/drivers/location → updates Redis GeoSet with fresh lat/lon — O(log N) operation

7 **Rider Gets Fare Estimate**

POST /api/pricing/estimate → Haversine distance + Redis surge multiplier → returns fare in INR

8 **Rider Requests a Ride**

POST /api/rides/request → saves ride (status=REQUESTED), sets Redis key with 60s TTL, publishes ride.requested to Kafka

9 **Kafka: ride.requested Event Fires**

driver-service consumes → queries Redis GEORADIUS for nearby drivers → notifies them pricing-service consumes → increments demand counter for the zone

10 **Driver Accepts the Ride (within 60s)**

POST /api/rides/{rideId}/accept → checks Redis TTL (fails if expired), sets status=DRIVER_ASSIGNED, deletes Redis key, publishes ride.accepted

11 **Kafka: ride.accepted Event Fires**

driver-service consumes → sets driver status=ON_RIDE notification-service → notifies rider that driver is on the way

12 **Driver Starts the Ride**

POST /api/rides/{rideId}/start → sets status=STARTED, records startedAt timestamp, publishes ride.started

13 **Kafka: ride.started Event Fires**

notification-service → notifies rider that trip has started

14 **Driver Completes the Ride**

POST /api/rides/{rideId}/complete → sets status=COMPLETED, records completedAt, sets finalFare, publishes ride.completed

15 **Kafka: ride.completed Event Fires**

payment-service consumes → idempotency check → creates Stripe PaymentIntent → publishes payment.success or payment.failed notification-service → notifies rider of trip summary

16 **Saga: payment.failed Compensation (if payment fails)**

ride-service PaymentEventConsumer → sets ride status=PAYMENT_FAILED (compensating transaction) notification-service → notifies rider of payment failure

17 **Rider Checks Payment Status**

GET /api/payments/ride/{rideId} → returns payment status: PENDING / SUCCESS / FAILED / REFUNDED

18 **Rider Views Ride History**

GET /api/rides/my-rides → returns all rides for the rider, newest first

Section 2 — User Service APIs (port 8081)

Handles rider registration, authentication, and profile. JWT tokens from this service are valid for 24 hours.

POST /api/users/register

Register a new rider account. Returns a JWT token immediately — no need to call login separately after registration.

Request Body

```
{
"email": "rider@test.com",
"password": "pass12345",
"fullName": "John Doe",
"phone": "9876543210"
}
```

Response

```
{
"token": "eyJhbGciOiJIUzI1NiJ9...",
"userId": "uuid-xxx",
"email": "rider@test.com"
}
```

Note: *Password min 8 chars. Phone must be exactly 10 digits. Email must be unique.*

POST /api/users/login

Login with email and password. Returns a fresh JWT token. Use this for subsequent app sessions after registration.

Request Body

```
{
"email": "rider@test.com",
"password": "pass12345"
}
```

Response

```
{
"token": "eyJhbGciOiJIUzI1NiJ9...",
"userId": "uuid-xxx",
"email": "rider@test.com"
}
```

Note: *Returns HTTP 401 if email not found or password doesn't match.*

GET /api/users/me

Get the authenticated rider's profile. The Gateway extracts userId from the JWT and injects it as X-User-Id header — no userId needed in the request.

Request Body

```
Headers:
Authorization: Bearer
```

Response

```
{
"id": "uuid-xxx",
"email": "rider@test.com",
"fullName": "John Doe",
"phone": "9876543210",
"role": "RIDER"
}
```

Section 3 — Driver Service APIs (port 8082)

Manages driver lifecycle: registration, login, real-time GPS tracking via Redis GeoSet, and availability status. Location updates should be called every 5 seconds by the driver app.

POST /api/drivers/register

Register a new driver with vehicle details. Returns a JWT token immediately. Vehicle number must follow Indian format (e.g. MH01AB1234).

Request Body

```
{
"email": "driver@test.com",
"password": "pass12345",
"fullName": "Ram Kumar",
"phone": "9123456789",
"vehicleNumber": "MH01AB1234",
"vehicleType": "SEDAN"
}
```

Response

```
{
"token": "eyJhbGciOiJIUzI1NiJ9...",
"driverId": "uuid-yyy",
"email": "driver@test.com",
"vehicleNumber": "MH01AB1234",
"status": "OFFLINE"
}
```

Note: *vehicleType must be one of: SEDAN, SUV, AUTO, BIKE. vehicleNumber format: MH01AB1234.*

POST /api/drivers/login

Authenticate a driver. Returns a JWT token valid for 24 hours. Use the token for all subsequent driver API calls.

Request Body

```
{
"email": "driver@test.com",
"password": "pass12345"
}
```

Response

```
{
"token": "eyJhbGciOiJIUzI1NiJ9...",
"driverId": "uuid-yyy",
"status": "OFFLINE"
}
```

Note: *This endpoint is public (no token needed). The JWT contains role=DRIVER.*

POST /api/drivers/online

Set driver status to ONLINE and register their starting location in Redis GeoSet. Driver must be ONLINE to receive ride requests. This is the first call a driver makes when starting their shift.

Request Body

```
Headers: Authorization: Bearer
{
"latitude": 19.0760,
"longitude": 72.8777
}
```

Response

```
{
"id": "uuid-yyy",
"status": "ONLINE",
"email": "driver@test.com"
}
```

Note: *Latitude must be -90 to 90. Longitude must be -180 to 180. Driver added to Redis GeoSet via GEOADD.*

POST /api/drivers/location

Update driver's current GPS coordinates in Redis GeoSet. Should be called every 5 seconds by the driver app while ONLINE. Uses Redis GEOADD — O(log N) operation, handles thousands of concurrent drivers efficiently.

Request Body

```
Headers: Authorization: Bearer
{
"latitude": 19.0810,
"longitude": 72.8820
}
```

Response

```
(204 No Content)
```

POST /api/drivers/offline

Set driver status to OFFLINE and remove them from Redis GeoSet. They will no longer appear in nearby driver searches. Call this when the driver ends their shift.

Request Body

```
Headers: Authorization: Bearer
```

Response

```
{
"id": "uuid-yyy",
"status": "OFFLINE"
}
```

GET /api/drivers/me

Get the authenticated driver's profile including current status (ONLINE/OFFLINE/ON_RIDE).

Request Body

```
Headers: Authorization: Bearer
```

Response

```
{
"id": "uuid-yyy",
"email": "driver@test.com",
"fullName": "Ram Kumar",
"vehicleNumber": "MH01AB1234",
"vehicleType": "SEDAN",
"status": "ONLINE"
}
```

GET /api/drivers/nearby?lat=19.076&lon;=72.877&radius;=5.0

Find all ONLINE drivers within a given radius (km) of a location. Uses Redis GEORADIUS — returns results in milliseconds regardless of how many drivers are registered. Default radius is 5km.

Request Body

```
Query params: lat, lon, radius (optional, default 5.0km)
```

Response

```
[
"driver-uuid-001",
"driver-uuid-002",
"driver-uuid-003"
]
```

> ***Note:*** *Returns driver IDs only. Call /api/drivers/{id} to get full profile.*

Section 4 — Ride Service APIs (port 8083)

Manages the full ride lifecycle. Each status transition publishes a Kafka event which other services consume. Rides expire automatically after 60 seconds if no driver accepts (Redis TTL).

Also: CANCELLED (from REQUESTED or DRIVER_ASSIGNED) | PAYMENT_FAILED (from COMPLETED when payment fails — Saga compensating state)

POST /api/rides/request

Rider requests a ride. Internally: saves ride with status=REQUESTED → stores ride key in Redis with 60s TTL → publishes ride.requested to Kafka. Fare is estimated using Haversine distance: base ■30 + ■12/km.

Request Body

```
Headers: Authorization: Bearer
{
"pickupLat": 19.0760, "pickupLon": 72.8777,
"dropLat": 19.0880, "dropLon": 72.8950,
"pickupAddress": "Bandra West",
"dropAddress": "Andheri East"
}
```

Response

```
{
"rideId": "ride-uuid-001",
"status": "REQUESTED",
"estimatedFare": 150.00,
"pickupAddress": "Bandra West",
"dropAddress": "Andheri East",
"createdAt": "2024-01-15T10:30:00"
}
```

***Note:** The 60s Redis TTL starts NOW. If no driver accepts within 60s, the ride expires and acceptRide will fail.*

POST /api/rides/{rideId}/accept

Driver accepts a ride request. Checks Redis TTL — if the 60s window has passed, returns error. On success: assigns driver, deletes Redis key, publishes ride.accepted. Two drivers cannot accept the same ride (status check prevents it).

Request Body

```
Headers: Authorization: Bearer
Path param: rideId
```

Response

```
{
"rideId": "ride-uuid-001",
"status": "DRIVER_ASSIGNED",
"driverId": "driver-uuid-yyy"
}
```

***Note:** Returns 400 if Redis TTL expired ('Ride request has expired') or ride already accepted ('Ride is no longer available').*

POST /api/rides/{rideId}/start

Driver starts the ride (picks up the rider). Sets status=STARTED, records startedAt timestamp, publishes ride.started event. Only the assigned driver can start — other drivers get Unauthorized error.

Request Body

```
Headers: Authorization: Bearer
Path param: rideId
```

Response

```
{
"rideId": "ride-uuid-001",
"status": "STARTED",
"startedAt": "2024-01-15T10:35:00"
}
```

POST /api/rides/{rideId}/complete

Driver marks ride as completed. Sets status=COMPLETED, records completedAt, sets finalFare = estimatedFare, then publishes ride.completed to Kafka. This event triggers the entire payment flow asynchronously.

Request Body

```
Headers: Authorization: Bearer
Path param: rideId
```

Response

```
{
"rideId": "ride-uuid-001",
"status": "COMPLETED",
"finalFare": 150.00,
"completedAt": "2024-01-15T10:55:00"
}
```

Note: *After this call, payment-service picks up the ride.completed Kafka event and charges the rider via Stripe.*

POST /api/rides/{rideId}/cancel

Cancel a ride. Can be called by rider or driver. Cannot cancel a ride that is already STARTED. Publishes ride.cancelled event — driver-service sets driver back to ONLINE.

Request Body

```
Headers: Authorization: Bearer
Path param: rideId
```

Response

```
{
"rideId": "ride-uuid-001",
"status": "CANCELLED"
}
```

Note: *Returns 400 if ride is in STARTED status ('Cannot cancel a ride in progress').*

GET /api/rides/{rideId}

Get full details of a specific ride by ID.

Request Body

```
Headers: Authorization: Bearer
Path param: rideId
```

Response

```
{
"rideId": "...",
"status": "COMPLETED",
"riderId": "...",
"driverId": "...",
"estimatedFare": 150.00,
"finalFare": 150.00
}
```

GET /api/rides/my-rides

Get all rides for the authenticated rider, ordered newest first. Use for ride history screen.

Request Body

```
Headers: Authorization: Bearer
```

Response

```
[
{ "rideId": "...", "status": "COMPLETED", "finalFare": 150.00 },
{ "rideId": "...", "status": "CANCELLED" }
]
```

GET /api/rides/driver/active

Get active rides (status=DRIVER_ASSIGNED) for the authenticated driver.

Request Body

```
Headers: Authorization: Bearer
```

Response

```
[
{ "rideId": "...", "status": "DRIVER_ASSIGNED", "riderId": "...",
"pickupAddress": "..." }
]
```

Section 5 — Pricing Service APIs (port 8085)

Calculates fares and manages real-time surge pricing. Demand and supply counters are tracked per city zone in Redis. A @Scheduled job recalculates the surge multiplier every 30 seconds. Max surge is 3x.

POST /api/pricing/estimate

Get a fare estimate before booking. Calculates Haversine distance between pickup and drop, fetches current surge multiplier for the zone from Redis, returns full breakdown. Formula: (base ■30 + ■12/km) × surge multiplier.

Request Body

```
{
"pickupLat": 19.0760, "pickupLon": 72.8777,
"dropLat": 19.0880, "dropLon": 72.8950,
"zone": "zone_center"
}
```

Response

```
{
"estimatedFare": 165.00,
"surgeMultiplier": 1.5,
"distanceKm": 7.2,
"durationMin": 21.6,
"zone": "zone_center",
"currency": "INR"
}
```

Note: *zone is optional — defaults to 'zone_center'. Surge 1.5 means 50% extra charge due to high demand.*

GET /api/pricing/estimate?distance=10&duration;=20&zone;=zone_center

Quick fare estimate by providing distance and duration directly (no coordinates needed).

Request Body

```
Query params: distance (km), duration (min, optional), zone (optional)
```

Response

```
{
"estimatedFare": 150.00,
"surgeMultiplier": 1.0,
"distanceKm": 10.0,
"currency": "INR"
}
```

GET /api/pricing/surge/{zone}

Get the current surge multiplier for a specific zone. Updated every 30 seconds. surgeActive=true means multiplier > 1.0 (higher demand than supply in this zone).

Request Body

```
Path param: zone (e.g. zone_center, zone_north, zone_airport)
```

Response

```
{
"zone": "zone_center",
"multiplier": 2.0,
"surgeActive": true
}
```

Note: *Multiplier 1.0 = normal pricing. 2.0 = 2x surge. Max is 3.0x. Recalculated every 30s by @Scheduled job.*

Section 6 — Payment Service APIs (port 8084)

Payment is triggered asynchronously via Kafka — you don't call it directly to charge. When ride.completed fires, payment-service processes the Stripe charge automatically. Two-layer idempotency prevents double-charging on retry.

GET /api/payments/ride/{rideId}

Check the payment status for a completed ride. Poll this after ride completion to know if payment succeeded.

Request Body

```
Headers: Authorization: Bearer
Path param: rideId
```

Response

```
{
"paymentId": "pay-uuid-001",
"rideId": "ride-uuid-001",
"amount": 150.00,
"currency": "inr",
"status": "SUCCESS",
"stripePaymentIntentId": "pi_xxx"
}
```

Note: *Status values: PENDING (being processed) | SUCCESS (charged) | FAILED (charge failed) | REFUNDED*

GET /api/payments/{paymentId}

Get full payment details by payment ID.

Request Body

```
Headers: Authorization: Bearer
```

Response

```
{
"paymentId": "pay-uuid-001",
"status": "SUCCESS",
"amount": 150.00
}
```

POST /api/payments/webhook

Stripe webhook endpoint — called by Stripe to confirm payment status asynchronously. Do NOT call this yourself. Stripe calls it after processing a PaymentIntent. Validates Stripe-Signature header to prevent spoofing.

Request Body

```
Called by Stripe (not by your app)
Headers: Stripe-Signature: t=...,v1=...
```

Response

```
200 OK: "OK"
400 Bad Request: webhook validation error
```

Note: *For local testing, use 'stripe listen --forward-to localhost:8084/api/payments/webhook' to forward Stripe events.*

POST /api/payments/retry/{rideId}

Manually retry payment for a ride whose payment failed. Idempotency is enforced — safe to call multiple times.

Request Body

```
Headers: Authorization: Bearer
Path param: rideId
```

Response

```
"Retry initiated"
```

POST /api/payments/refund

Request a refund for a completed ride. Requires a reason (min 10 characters).

Request Body

```
Headers: Authorization: Bearer
{
"rideId": "ride-uuid-001",
"reason": "Driver took wrong route and overcharged"
}
```

Response

```
"Refund initiated for ride: ride-uuid-001"
```

> ***Note:*** *Reason must be between 10 and 500 characters.*

Section 7 — Kafka Event Flows

Events are the backbone of this system. Services communicate via Kafka topics — not direct HTTP calls. This means each service is independent: it can fail, restart, or scale without affecting others.

Every Kafka Topic and Who Processes It

Producer	Topic	Consumers + What They Do
Ride Service	→ **ride.requested**	Driver Service → finds nearby drivers via GEORADIUS, notifies them Pricing Service → increments demand counter for the zone
Ride Service	→ **ride.accepted**	Driver Service → sets driver status = ON_RIDE Notification Service → tells rider 'Driver is on the way'
Ride Service	→ **ride.started**	Notification Service → tells rider 'Your trip has started'
Ride Service	→ **ride.completed**	Payment Service → idempotency check → Stripe PaymentIntent → payment.success or payment.failed Notification Service → sends trip summary to rider
Ride Service	→ **ride.cancelled**	Driver Service → sets driver status = ONLINE (available again) Notification Service → notifies both rider and driver
Payment Service	→ **payment.success**	Ride Service (PaymentEventConsumer) → logs confirmation (ride stays COMPLETED) Notification Service → 'Payment successful, receipt sent'
Payment Service	→ **payment.failed**	Ride Service (PaymentEventConsumer) → sets ride = PAYMENT_FAILED (Saga compensating transaction) Notification Service → 'Payment failed, please retry'

The Saga Pattern — Why ride.completed → payment.failed → PAYMENT_FAILED

In a monolith, you'd wrap ride completion and payment in one DB transaction. In microservices that's impossible — each service has its own database. The Saga pattern solves this: each service does its local work and publishes an event. If a step fails, a compensating event rolls back the previous step.

Step	Service	Action	Status
1	ride-service	Driver completes ride	ride → COMPLETED
2	payment-service	Consumes ride.completed	payment → PENDING
3a	payment-service	Stripe SUCCESS → publishes payment.success	payment → SUCCESS
3b	payment-service	Stripe FAILED → publishes payment.failed	payment → FAILED
4b	ride-service	Consumes payment.failed (compensating tx)	ride → PAYMENT_FAILED

Section 8 — Auth, Rate Limiting & Validation

JWT Authentication Flow

All authenticated endpoints require Authorization: Bearer <token> in the header. The API Gateway validates the token and injects user context — downstream services never re-validate JWT.

Step	Where	What Happens
1	Login/Register API	Service generates JWT with userId (subject) + role claim, signed with HS256
2	Client stores token	Store in memory/secure storage — valid for 24 hours
3	API Gateway (JwtAuthFilter)	Extracts token from Authorization header, validates HS256 signature
4	API Gateway (JwtAuthFilter)	Injects X-User-Id and X-User-Role headers into the forwarded request
5	Downstream service	Reads X-User-Id from header — no DB or JWT library call needed

Public Endpoints (no token required)

- `POST /api/users/register`
- `POST /api/users/login`
- `POST /api/drivers/register`
- `POST /api/drivers/login`
- `GET /api/drivers/nearby`
- `GET /actuator/**`

Rate Limiting

Redis token bucket rate limiting is applied at the API Gateway. Each user gets their own bucket (keyed by X-User-Id). Unauthenticated requests are limited by IP address. Exceeded requests receive HTTP 429 Too Many Requests.

Endpoint	Rate	Burst	Why
/api/users/register, /login	3 req/s	5	Brute-force attack protection
/api/drivers/register, /login	3 req/s	5	Brute-force attack protection
All other routes (default)	10 req/s	20	Normal API usage headroom

Input Validation Rules

All request bodies are validated before hitting service logic. Invalid input returns HTTP 400 with a field-by-field error message.

Field	Rule	Error if violated
email	Valid email format, not blank	Must be a valid email address
password	Min 8 characters	Password must be at least 8 characters
fullName	2-100 characters	Full name must be between 2 and 100 chars
phone	Exactly 10 digits	Phone must be a 10-digit number
vehicleNumber	Format: MH01AB1234	Vehicle number must be in format MH01AB1234

vehicleType	SEDAN / SUV / AUTO / BIKE	Vehicle type must be one of: SEDAN, SUV, AUTO, BIKE
latitude	-90.0 to 90.0, not null	Latitude must be between -90 and 90
longitude	-180.0 to 180.0, not null	Longitude must be between -180 and 180
pickupAddress	Not blank	Pickup address is required
dropAddress	Not blank	Drop address is required
refund reason	10-500 characters	Reason must be between 10 and 500 characters

Section 9 — Quick Reference Cheatsheet

METHOD	Endpoint	Auth?	Service	What it does
POST	/api/users/register	No	user	Register rider, get JWT
POST	/api/users/login	No	user	Login rider, get JWT
GET	/api/users/me	Yes	user	Get rider profile
POST	/api/drivers/register	No	driver	Register driver, get JWT
POST	/api/drivers/login	No	driver	Login driver, get JWT
GET	/api/drivers/me	Yes	driver	Get driver profile
POST	/api/drivers/online	Yes	driver	Go online, register GPS
POST	/api/drivers/offline	Yes	driver	Go offline, remove from GeoSet
POST	/api/drivers/location	Yes	driver	Update GPS (every 5s)
GET	/api/drivers/nearby	No	driver	Find nearby ONLINE drivers
POST	/api/pricing/estimate	Yes	pricing	Fare estimate with surge
GET	/api/pricing/estimate	Yes	pricing	Fare estimate by distance
GET	/api/pricing/surge/{zone}	Yes	pricing	Current surge multiplier
POST	/api/rides/request	Yes	ride	Request a ride (60s TTL)
POST	/api/rides/{rideId}/accept	Yes	ride	Driver accepts ride
POST	/api/rides/{rideId}/start	Yes	ride	Driver starts ride
POST	/api/rides/{rideId}/complete	Yes	ride	Driver completes ride → triggers payment
POST	/api/rides/{rideId}/cancel	Yes	ride	Cancel ride
GET	/api/rides/{rideId}	Yes	ride	Get ride details
GET	/api/rides/my-rides	Yes	ride	Rider's ride history
GET	/api/rides/driver/active	Yes	ride	Driver's active rides
GET	/api/payments/ride/{rideId}	Yes	payment	Payment status for ride
GET	/api/payments/{paymentId}	Yes	payment	Payment by ID
POST	/api/payments/retry/{rideId}	Yes	payment	Retry failed payment
POST	/api/payments/refund	Yes	payment	Request refund
POST	/api/payments/webhook	No	payment	Stripe webhook (Stripe calls this)

All requests go through http://localhost:8080 (API Gateway) • Swagger UI available at http://localhost:{port}/swagger-ui.html for each service • Kafka UI: http://localhost:8090 • Zipkin: http://localhost:9411 • Grafana: http://localhost:3000